БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Факультет прикладной математики и информатики
Кафедра технологий программирования

Лабораторная работа №4
Методы и средства оркестрации в многоконтейнерных системах

По дисциплине «Непрерывное интегрирование и сборка программного обеспечения»

Методические указания по выполнению лабораторной работы

Подготовила:
Давидовская М. И.,
Ст. преподаватель кафедры ТП

Минск, 2026 г.

Содержание

Цель работы....3
Задачи работы....3
Задания....3
Методические указания....3
Критерии оценивания....3
Содержание отчета....3
Задание 1. Подготовка окружения для локального кластера....4
Задание 2. Знакомство с ресурсами kubernetes....5
Задание 3. Работа с Kubernetes....6
Контрольные вопросы....6

Цель работы

Изучение создание кластера и работу с kubernets

Задачи работы

1. Настроить окружение kubernets.
2. Выполнить задания

Задания

Методические указания

Все результаты лабораторной работы должны быть опубликованы в git-репозитории, ссылка на который доступна в курсе «Непрерывное интегрирование и сборка программного обеспечения»

Критерии оценивания

Для групп 10-12 выполнить все 3 задания, для группы 13 — задания 1-2.

Содержание отчета

1. Цель работы.
2. Вариант задания.
3. Код приложений, конфигурационных файлов.
4. Ответы на контрольные вопросы.

Отчет должен быть опубликован в git-репозитории на github. Все результаты лабораторной работы должны быть опубликованы в git-репозитории, ссылка на который доступна в курсе «Непрерывное интегрирование и сборка программного обеспечения».

В файле Readme проекта на github должна быть ссылка на отчёт. Отчет опубликовать во внешнем хранилище или в репозитории в каталоге /docs. Если в лабораторной работе необходимо написать программу/ы, то отчёт должен результаты тестов по каждой программе и ответы на контрольные вопросы.

Пример оформления файла Readme может быть таким:

```
# Overview

Report on LabRabota1.

# Usage

// Заменить <<link>> и <<folder>> на соответствующие ссылки/названия
To check, please, preview report by <<link>> and source files
in <<folder>>.

# Author

Your name and group number.

# Additional Notes

// СКОПИРОВАТЬ И ВСТАВИТЬ ССЫЛКУ НА СВОЙ РЕПОЗИТОРИЙ,НАПРИМЕР
https://github.com/maryiad/lab3-task1-gr16-david
```

Каждая лабораторная работа содержит тексты задач и контрольные вопросы, ответы на которые проверяются преподавателем при приёме работы у студента.

Выполнение студентом лабораторной работы и сдача её результатов преподавателю происходит следующим образом:

1. Студент выполняет разработку программ.

2. В ходе разработки студент обязан следовать указаниям к данной задаче (в случае их наличия). Исходные тексты программ следует разрабатывать в соответствии с требованиями к оформлению, приведёнными в приложении.

3. Студент выполняет самостоятельную проверку исходного текста каждой разработанной программы и правильности её работы, а также свои знания по теме лабораторной работы.

Задание 1. Подготовка окружения для локального кластера

Для работы нам понадобятся утилита kubect и локальный кластер kubernetes. Рассматриваем установку кластера в виртуальной машине,

поэтому рекомендуется использовать утилиту `kind`. Утилита `kind` требует наличия Docker в системе. Можно использовать и отдельный `minikube`.

1. Изучить статьи раздела «**Кластер Kubernetes**»:
    1. «Kubernetes»
    2. «Дистрибутивы и инструменты Kubernetes»
2. Изучить материалы семинара «**Кластер Kubernetes**»:
    1. «Подготовка окружения» (без Terraform)
    2. «Разворачивание кластера при помощи kind и kubectl»
3. Изучить Главу 16 «Introducing Kubernetes» из книги The_Ultimate_Docker_Container_Book_3rd_Edition_(2023)
4. Изучить Главу 15 «**Introducing Kubernetes**» The_Ultimate_Docker_Container_Book_4th_Edition_(2026).
5. Установить необходимые утилиты для работы с кластерами kubernets.
6. Создать кластер (см. примеры из п. 1 и 3 текущего задания).
7. Продемонстрировать работающий кластер.

Задание 2. Знакомство с ресурсами kubernetes

Все объекты в kubernetes — это ресурсы. Чтобы получить список ресурсов используется команда **kubectl get**. Чтобы посмотреть список всех возможных ресурсов в кластере с их псевдонимами используется команда **kubectl api-resources**. Например, команда **kubectl get namespaces** или **kubectl get ns** выведет список пространств имен, по которым сгруппированы остальные ресурсы.

Следующий важный ресурс в kubernetes — **POD**. Pod — единица работы в kubernetes, pod создается, запускается на подходящей ноде, работает и завершается. Pod может состоять из нескольких контейнеров (хотя чаще все же из одного), которые гарантировано запускаются вместе на одной ноде.

1. Изучить статьи раздела «**Кластер Kubernetes**»:
    1. «Kubernetes»
    2. «Дистрибутивы и инструменты Kubernetes»
2. Изучить материалы семинара «**Кластер Kubernetes**»:
    1. «Подготовка окружения» (без Terraform)
    2. «Разворачивание кластера при помощи kind и kubectl»

3. Изучить Главу 16 «Introducing Kubernetes» из книги The_Ultimate_Docker_Container_Book_3rd_Edition_(2023)
4. Изучить Главу 15 «**Introducing Kubernetes**» The_Ultimate_Docker_Container_Book_4th_Edition_(2026).
5. Изучить список pods в kubernetes. Объяснить, зачем нужны эти компоненты и описать в отчете.
6. Создать том и продемонстрировать его монтирование в pod (см. примеры в Главе 16. «Introducing Kubernetes» из книги The_Ultimate_Docker_Container_Book_3rd_Edition_(2023) или в Главе 15 «Introducing Kubernetes» из книги The_Ultimate_Docker_Container_Book_4th_Edition_(2026)).
7. Продемонстрировать работу с ReplicaSet, коллекцией pods (см. примеры в Главе 16 «Introducing Kubernetes» из книги The_Ultimate_Docker_Container_Book_3rd_Edition_(2023) или или в Главе 15 «Introducing Kubernetes» из книги The_Ultimate_Docker_Container_Book_4th_Edition_(2026)).

Задание 3. Работа с Kubernetes

1. Изучить статьи раздела «**Кластер Kubernetes**»:
    1. «Kubernetes»
    2. «Дистрибутивы и инструменты Kubernetes»
2. Изучить материалы семинара «**Кластер Kubernetes**»:
    1. «Подготовка окружения» (без Terraform)
    2. «Разворачивание кластера при помощи kind и kubectl»
3. Изучить Главу 16 «Introducing Kubernetes» из книги The_Ultimate_Docker_Container_Book_3rd_Edition_(2023)
4. Изучить Главу 15 «**Introducing Kubernetes**» The_Ultimate_Docker_Container_Book_4th_Edition_(2026).
5. Изучить Главу 17. «Deploying, Updating, and Securing an Application with Kubernetes» из книги The_Ultimate_Docker_Container_Book_3rd_Edition_(2023)
6. Изучить Главу 16. «Deploying, Updating, and Securing an Application with Kubernetes» из книги The_Ultimate_Docker_Container_Book_4th_Edition_(2026)
7. Проиллюстрировать развертывание приложения (deployment) в кластере Kubernetes.

Контрольные вопросы

1. Что такое Kubernetes и зачем он нужен в работе с контейнерами?
2. Объясните, в чём отличие «запустить контейнеры вручную через Docker» от «управлять ими в Kubernetes».
3. Приведите 1–2 примера задач, где оркестрация экономит время.
4. Назовите основные компоненты кластера Kubernetes и опишите, за что каждый отвечает.
5. Как создать под с помощью YAML-манифеста? Перечислите обязательные поля (например, apiVersion, kind, metadata, spec) и объясните, что они задают.
6. Приведите YAML-файл (10–15 строк) для простого пода с одним контейнером и прокомментируйте 3–4 ключевых поля. Что произойдёт, если одно из обязательных полей пропустить?
7. Что такое сервисы в Kubernetes и зачем они нужны? Объясните разницу между ClusterIP и Headless-сервисом на примере.
8. Чем отличаются сервисы NodePort и LoadBalancer? Опишите пошагово, как внешний пользователь попадает в под в каждом случае.
9. Укажите, какие порты задействованы и какие ограничения есть у NodePort.
10. Приведите простой пример, когда NodePort достаточно, а когда нужен именно LoadBalancer.
11. Что такое Ingress и какую проблему он решает? Приведите краткий пример Ingress-манифеста с одним правилом (путь + хост) и объясните, как он направляет трафик в нужный сервис.
12. Сравните Deployment, StatefulSet и DaemonSet по трём критериям: цель использования, как создаются поды (упорядоченно или нет), к чему привязаны (к узлам или нет). Для каждого типа приведите по одному простому примеру из практики (например, веб-приложение, база данных, агент логирования) и скажите, что произойдёт при масштабировании до 3 реплик.
13. В чём разница между Job и CronJob? Приведите по одному примеру задачи для каждого. Как проверить, что задача выполнилась, и где смотреть результат (статус, логи)?
14. Для чего нужны ConfigMap и Secret? Покажите 2 способа, как передать данные в под. Когда и какой способ удобнее использовать?

15. Приведите по короткому YAML-фрагменту для ConfigMap и Secret и объясните, почему пароли нельзя хранить в ConfigMap.
16. Перечислите 8–10 основных команд kubectl, которые чаще всего используются, и сгруппируйте их по задачам: посмотреть состояние, создать/обновить, удалить, отладить.
17. Что такое пространства имён (namespaces) и зачем они используются? Приведите 2–3 примера, где разделение по namespaces полезно (например, dev/stage/prod, разные команды).
18. Как создать новое пространство имён и как выполнять команды только в нём?
19. Зачем нужны пробы (probes): livenessProbe, readinessProbe и startupProbe?
20. Что такое Persistent Volume (PV) и Persistent Volume Claim (PVC)? Как выделяется место под том (хранилище) в Kubernetes?
21. Как Kubernetes реагирует, если под или узел «упали»? Опишите 2–3 механизма самовосстановления.
22. Как обновлять приложение без простоя? Объясните, что такое rolling update и какие параметры в Deployment за это отвечают?
23. Как запустить обновление приложения в кластере kubernetes и как следить на прогрессом?